**Interview question on inheritence , abstract ,interace and super**

**1. Inheritance Interview Questions**

- **What is inheritance in Java?**
- **Why is inheritance used?**
- **What are the types of inheritance in Java?**
- **Why Java does not support multiple inheritance with classes?**
- **What is the difference between single, multilevel, and hierarchical inheritance?**
- **What are the advantages and disadvantages of inheritance?**
- **What is method overriding in inheritance?**
- **What is the IS-A relationship?**

**2. super Keyword Interview Questions**

- **What is the super keyword in Java?**
- **Where can we use the super keyword?**
- **How do you access parent class variables using super?**
- **How do you call parent class methods using super?**
- **Difference between this and super**

**3. super() Constructor Questions**

- **What is super() in Java?**
- **When is super() called?**
- **Why is super() used in constructors?**
- **What happens if we don't explicitly call super()?**
- **Can we call parameterized super()?**
- **Can we use both this() and super() in same constructor?**

**4. Abstract Class Interview Questions**

- **What is an abstract class?**
- **Why do we use abstract classes?**
- **Can we create an object of an abstract class?**

**What is an abstract method?**

**Can abstract class have constructors?**

**Can abstract class have both abstract and non-abstract methods?**

**What is the difference between abstract class and concrete class?**

**5. Interface Interview Questions**

**What is an interface in Java?**

**Why do we use interfaces?**

**Difference between interface and abstract class**

**Can an interface have methods with body? (default/static methods)**

**Can an interface have variables?**

**What is multiple inheritance using interfaces?**

**What is a functional interface?**

**6. Abstract Class vs Interface (Very Important)**

**Difference between abstract class and interface**

**When should you use abstract class vs interface?**

**Can a class implement multiple interfaces?**

**Can an interface extend another interface?**

**7. Tricky / Frequently Asked Questions**

**Can we override static methods in inheritance?**

**Can constructors be inherited?**

**What happens if parent and child class have same method?**

**What is dynamic method dispatch?**

**Can abstract class implement interface?**

**Can interface extend a class?**

**8. Coding-Based Questions**

**Write a program to demonstrate inheritance**

**Use super keyword to access parent class members**

**Demonstrate constructor chaining using super()**

**Create an abstract class with abstract method**

**Interview question on final keyword**

---

**1. Final Variable Interview Questions**

**What is a final variable in Java?**

**Can a final variable be initialized later? (Explain blank final variable)**

**Can a final variable be changed after initialization?**

**Difference between final variable and constant**

**Can instance variables and static variables be final?**

**Example use case of a final variable**

**2. Final Method Interview Questions**

**What is a final method in Java?**

**Can a final method be overridden? Why or why not?**

**Can a final method be static?**

**Why/when do we declare a method as final?**

**Difference between final, abstract, and normal methods**

**3. Final Class Interview Questions**

**What is a final class in Java?**

**Can a final class be inherited? Why or why not?**

**Can a final class have final methods?**

**Example use case of final class**

**Difference between final class and abstract class**

**4. Combined / Conceptual Questions**

**What is the purpose of the final keyword in Java?**

**Can a final variable reference an object? Can the object itself be changed?**

**Difference between final variable, final method, and final class**

**Can a constructor be final? Why or why not?**

**Can you make a class final but its methods non-final?**

**5. Tricky / Frequently Asked Questions**

**What happens if a subclass tries to override a final method?**

**Can an interface have final methods?**

**Can a local variable inside a method be final?**

**What is the difference between final and immutable?**

**Interview question on static**

---

**1. Basic Static Keyword Questions**

**What does the static keyword mean in Java?**

**Which members of a class can be declared static? (variables, methods, blocks, nested classes)**

**What is a static variable? How is it different from instance variable?**

**What is a static method? How is it different from instance method?**

**Can a static method access instance variables? Why or why not?**

**Can a static method be overridden?**

**2. Static Block / Initialization Questions**

**What is a static block in Java?**

**When is a static block executed?**

**Can a class have multiple static blocks?**

**Difference between static block and constructor**

**3. Static Class / Nested Class Questions**

**What is a static nested class?**

**Difference between static nested class and inner (non-static) class**

**Can static nested class access instance members of outer class?**

**4. Advanced / Conceptual Questions**

**Can a static method use this or super keyword?**

**Can we declare main() as static? Why is it required?**

**Difference between static and final keywords**

**Can a static variable reference an object?**

**Difference between static variable and local variable in memory allocation**

**5. Tricky / Frequently Asked Questions**

**What happens if a static method calls a non-static method?**

**Can abstract classes have static methods?**

**Can interfaces have static methods?**

**Difference between class-level and instance-level members**

**What is memory allocation for static members vs instance members?**

**6. Interview Focus (Most Important)**

**Static variables and memory sharing across objects**

**Static methods vs instance methods (restrictions and usage)**

**Static block execution order**

**Main method as static**

**Interview question on exception handling**

---

**1. Basic Exception Handling Questions**

**What is an exception in Java?**

**What is the difference between error and exception?**

**What is the difference between checked and unchecked exceptions?**

**What are the main types of exceptions in Java?**

**What is the difference between Throwable, Exception, and Error?**

**2. Try-Catch-Finally Questions**

**What is the purpose of try, catch, and finally blocks?**

**Can a try block exist without catch?**

**Can a catch block exist without try?**

**What is the purpose of finally blocking?**

**What happens if an exception is thrown in the finally block?**

**3. Throws and Throw Questions**

**What is the difference between throw and throws?**

**Can a method throw multiple exceptions?**

**What happens if a checked exception is not handled or declared?**

**Can throw throw multiple exceptions?**

**4. Custom Exceptions Questions**

**What is a custom exception?**

**How do you create a user-defined exception in Java?**

**When should you create a custom exception?**

**Difference between extending Exception vs RuntimeException**

**5. Exception Propagation / Hierarchy Questions**

**What is exception propagation in Java?**

**How does exception propagation work in method calls?**

**What is the hierarchy of exceptions in Java?**

**What is the difference between Error, Exception, and RuntimeException?**

**6. Advanced / Conceptual Questions**

**What is the difference between checked vs unchecked exceptions?**

**What are best practices for exception handling?**

**Can we catch multiple exceptions in a single catch block?**

**What is the advantage of try-with-resources?**

**Can we re-throw an exception?**

**Difference between Exception and RuntimeException**

**7. Tricky / Frequently Asked Questions**

**What happens if an exception is thrown in the constructor?**

**Can finally block suppress an exception?**

**Difference between getMessage() and toString() in exceptions**

**Difference between throw and return in a catch block**

**Why should exceptions not be ignored?**

**8. Interview Focus (Most Important)**

**Checked vs Unchecked exceptions**

**Try-catch-finally behavior**

**throw vs throws**

**Custom exception creation**

**Exception propagation and hierarchy**

**Wrapper classes**

---

**Basic Questions**

**What are wrapper classes in Java?**

**Why do we need wrapper classes?**

**List all primitive types and their corresponding wrapper classes.**

**What is the difference between int and Integer?**

**Can wrapper classes be null?**

**Concept-Based Questions**

**What is autoboxing and unboxing?**

**When does Java perform autoboxing automatically?**

What happens if you unbox a null object?

Are wrapper classes mutable or immutable? Explain.

Why are wrapper classes used in collections like ArrayList?

Advanced Questions

What is Integer caching in Java?

What is the range of cached values in Integer?

Why does == behave differently for wrapper objects?

What is the difference between valueOf() and new Integer()?

Which is better: Integer.valueOf(10) or new Integer(10)? Why?

MCQ Questions

---

Q56

```
class Q56 {
   final int x;
   Q56() { x = 10; }
   Q56(int x) { this.x = x; }
   public static void main(String[] args) {
      Q56 obj = new Q56();
      System.out.println(obj.x);
   }
}
```

Q57.What will be output of given code?

```
class Q57 {
   static int x = 10;
   static { x += 5; }
   public static void main(String[] args) {
      System.out.println(x);
   }
```

```
}
```

**Q58.What will be output of given code?**

```
class Q58 {
   int x = 5;
   Q58() { this(10); }
   Q58(int x) { this.x = x; }
   public static void main(String[] args) {
      Q58 obj = new Q58();
      System.out.println(obj.x);
   }
}
```

**Q59.What will be output of given code?**

```
class Parent {
   Parent() { System.out.println("Parent"); }
}
class Child extends Parent {
   Child() { System.out.println("Child"); }
   public static void main(String[] args) { new Child(); }
}
```

**Q60.What will be output of given code?**

```
class Q60 {
   final void display() { System.out.println("Final"); }
}
class SubQ60 extends Q60 {
   // void display() { System.out.println("Override"); } // Uncomment?
   public static void main(String[] args) { new SubQ60().display(); }
}
```

**Q61.What will be output of given code?**

```
class Parent {
   static void show() { System.out.println("Parent"); }
}
class Child extends Parent {
   static void show() { System.out.println("Child"); }
   public static void main(String[] args) {
      Parent obj = new Child();
      obj.show();
   }
}
```

**Q62.What will be output of given code?**

```
class Parent {
   void show() { System.out.println("Parent"); }
}
class Child extends Parent {
   void show(int x) { System.out.println("Child"); }
   public static void main(String[] args) {
      Parent obj = new Child();
      // obj.show(5); // Uncomment?
   }
}
```

**Q63.What will be output of given code?**

```
class Q63 {
   void print(Object o) { System.out.println("Object"); }
   void print(String s) { System.out.println("String"); }
   public static void main(String[] args) {
      new Q63().print(null);
   }
}
```

**Q64.What will be output of given code?**

```
class Parent
 { void show()
   {
     System.out.println("Parent");
   }
   }
class Child extends
Parent
 {
   void show()
   { System.out.println("Child");
   }
 }
class GrandChild extends Child
 {
   void show()
   {
    System.out.println("GrandChild");
   }
  }
class Q64 {
    public static void main(String[] args) {
        Parent obj = new GrandChild();
        obj.show();
    }
}
```

**Q65.What will be output of given code?**

```
class Q65 {
```

```
   void test(int x)
          {
          System.out.println("int");
          }
   void test(long x)
          {
            System.out.println("long");
  }
   public static void main(String[] args) {
      Q65 obj = new Q65();
      obj.test(10);
   }
}
```

**Q66.What will be output of given code?**

```
interface I66
{
  void display();
}
class Q66 {
   public static void main(String[] args) {
      I66 obj = new I66() { public void display() { System.out.println("Anonymous"); } };
      obj.display();
   }
}
```

**Q67.What will be output of given code?**

```
abstract class Q67 {
   abstract void show();
}
```

```
class Test67 {
   public static void main(String[] args) {
      Q67 obj = new Q67()
                    {
                      void show()
                      {
                        System.out.println("Anonymous Abstract");
                      }
            };
      obj.show();
   }
}
```

**Q68.What will be output of given code?**

```
interface I70
{ void display();
}
abstract class Q70 implements I70 {
   public void display()
          {
            System.out.println("Implemented");
          }
   public static void main(String[] args)
          {   new Q70() {}.display();
          }
}
```

**Q71.What will be output of given code?**

```
  class Parent
```

```
 { void show()
   {
            System.out.println("Parent");
          }
 }
class Child extends Parent
 {
  void show()
  { System.out.println("Child");
  }
}
class Q71 {
   public static void main(String[] args) {
      Parent obj = new Child();
      ((Child)obj).show();
   }
}
```

**Q72.What will be output of given code?**

```
class Q72 {
   void display(int x)
          {
              System.out.println("int");
           }
   void display(Integer x)
          { System.out.println("Integer");
          }
   public static void main(String[] args)
          { new Q72().display(null);
          }
}
```

**Q73. What will be output of given code?**

```
class Q73 {
   static void show()
          {
            System.out.println("Static");
    }
   void show(int x)
          {
             System.out.println("Non-static");
          }
   public static void main(String[] args)
          { show();
          }
}
```

**Q74.What will be output of given code?**

```
class Outer {
   static int x = 10;
   static class Inner
          { void show()
            {
               System.out.println(x);
            }
   }
   public static void main(String[] args)
          {
             new Inner().show();
          }
```

```
}
```

**Q75. What will be output of given code?**

```
interface I75
 {
   int x = 5;
}
class Q75 implements I75 {
   static
           {
             System.out.println(x);
    }
   public static void main(String[] args)
           {
           }
}
```

**MCQ Question on Exception handling**

---

**Exception Handling MCQs (Q1–Q30)**

**Q1**

```
public class Q1 {
  public static void main(String[] args) {
     try {
        int a = 5/0;
     } catch (ArithmeticException e) {
        System.out.println("Caught");
     }
  }
}
```

**Q2**

```
public class Q2 {
  public static void main(String[] args) {
    try {
      int arr[] = new int[2];
      arr[3] = 10;
    } catch (ArrayIndexOutOfBoundsException e) {
      System.out.println("Exception");
    } finally {
      System.out.println("Finally");
    }
  }
}
```

**Q3**

```
public class Q3 {
  public static void main(String[] args) {
    try {
      System.out.println("Try");
    } finally {
      System.out.println("Finally");
    }
  }
}
```

**Q4**

```
public class Q4 {
  public static void main(String[] args) {
    try {
      throw new Exception("Test");
    } catch (ArithmeticException e) {
      System.out.println("Caught");
    } finally {
```

```
        System.out.println("Finally Block");
    }
  }
}
```

Q5

```
class Q5 {
  static void method() throws ArithmeticException {
    throw new ArithmeticException();
  }
  public static void main(String[] args) {
    try {
      method();
    } catch (ArithmeticException e) {
      System.out.println("Caught");
    }
  }
}
```

Q6

```
public class Q6 {
  public static void main(String[] args) {
    try {
      int a = 10/0;
    } catch (NullPointerException e) {
      System.out.println("Null");
    } finally {
      System.out.println("Finally");
    }
  }
}
```

Q7

```
class Q7 {
```

```
  static void check() throws Exception {
    throw new Exception("Error");
  }
  public static void main(String[] args) {
    try {
      check();
    } catch (Exception e) {
      System.out.println(e.getMessage());
    }
  }
}
```

**Q8**

```
class Q8 {
  public static void main(String[] args) {
    try {
      System.out.println("Try");
      return;
    } finally {
      System.out.println("Finally");
    }
  }
}
```

**Q9**

```
class Q9 {
  static void test() throws Exception {
    throw new Exception();
  }
  public static void main(String[] args) throws Exception {
    test();
    System.out.println("After Exception");
  }
```

```
}
```

**Q10**

```
class Q10 {
  public static void main(String[] args) {
    try {
      int a = 10/0;
    } catch (ArithmeticException e) {
      System.out.println("Catch");
      try {
        int b = 5/0;
      } catch (ArithmeticException ex) {
        System.out.println("Nested Catch");
      }
    } finally {
      System.out.println("Finally");
    }
  }
}
```

**Q11**

```
class Q11 {
  public static void main(String[] args) {
    try {
      int arr[] = new int[2];
      arr[5] = 10;
    } catch (ArithmeticException e) {
      System.out.println("A");
    } catch (ArrayIndexOutOfBoundsException e) {
      System.out.println("B");
    } finally {
      System.out.println("C");
    }
```

```
  }
}
```

**Q12**

```
class Q12 {
  public static void main(String[] args) {
    try {
      System.out.println("Try");
    } catch (Exception e) {
      System.out.println("Catch");
    }
    System.out.println("After Try-Catch");
  }
}
```

**Q13**

```
class Q13 {
  public static void main(String[] args) {
    try {
      throw new NullPointerException("Null");
    } catch (RuntimeException e) {
      System.out.println(e.getMessage());
    } finally {
      System.out.println("Finally");
    }
  }
}
```

**Q14**

```
class Q14 {
  public static void main(String[] args) {
    try {
      System.out.println(10/0);
    } finally {
```

```
        System.out.println("Finally always executes");
    }
  }
}
```

**Q15**

```
class Q15 {
  static void divide(int a, int b) throws ArithmeticException {
    if(b==0) throw new ArithmeticException("Divide by zero");
    System.out.println(a/b);
  }
  public static void main(String[] args) {
    try {
      divide(10,0);
    } catch(Exception e) {
      System.out.println(e.getMessage());
    }
  }
}
```

**Q16**

```
class Q16 {
  public static void main(String[] args) {
    try {
      throw new Exception();
    } catch(Exception e) {
      System.out.println("Caught");
      return;
    } finally {
      System.out.println("Finally runs even after return");
    }
  }
}
```

**Q17**

```
class Q17 {
  public static void main(String[] args) {
    try {
      System.out.println("Try Block");
    } catch(Exception e) {
      System.out.println("Catch Block");
    } finally {
      System.out.println("Finally Block");
    }
    System.out.println("After Try-Catch-Finally");
  }
}
```

**Q18**

```
class Q18 {
  public static void main(String[] args) {
    try {
      System.exit(0);
    } finally {
      System.out.println("Finally?");
    }
  }
}
```

**Q19**

```
class Q19 {
  public static void main(String[] args) {
    try {
      int[] arr = new int[5];
      arr[5] = 10;
    } catch(Exception e) {
      System.out.println("Exception caught");
```

```
    } finally {
        System.out.println("Finally executed");
    }
  }
}
```

**Q20**

```
class Q20 {
  public static void main(String[] args) {
    try {
        throw new RuntimeException("Runtime");
    } catch(Exception e) {
        System.out.println("Exception");
    } finally {
        System.out.println("Finally Block");
    }
  }
}
```

## Quantitative Aptitude Questions

---

**PIPES & CISTERNS (10 MCQs)**

**Q1. A pipe can fill a tank in 6 hours. How much will it fill in 1 hour?**
**A) 1/3**
**B) 1/6**
**C) 1/12**
**D) 1/24**

**Answer: B) 1/6**
**Explanation:**
**If full tank = 1 unit**
**Filling in 6 hours → 1 hour work = 1/6**

---

**Q2. A pipe fills a tank in 8 hrs and another empties it in 12 hrs. Time to fill?**
**A) 20 hrs**
**B) 24 hrs**

**C) 16 hrs**
**D) 12 hrs**

**Answer: B) 24 hrs**
**Explanation:**
**Net work = (1/8 − 1/12) = (3−2)/24 = 1/24**
**Time = 24 hrs**

---

**Q3. Two pipes fill in 10 hrs and 15 hrs. Together?**
**A) 6 hrs**
**B) 5 hrs**
**C) 8 hrs**
**D) 12 hrs**

**Answer: A) 6 hrs**
**Explanation:**
**(1/10 + 1/15) = (3+2)/30 = 5/30 = 1/6 → 6 hrs**

---

**Q4. A leak empties tank in 20 hrs. Pipe fills in 10 hrs. Net?**
**A) 15 hrs**
**B) 20 hrs**
**C) 25 hrs**
**D) Cannot fill**

**Answer: B) 20 hrs**
**Explanation:**
**Net = (1/10 − 1/20) = 1/20 → 20 hrs**

---

**Q5. A tank filled in 12 hrs, but due to leak takes 16 hrs. Leak alone empties in?**
**A) 48 hrs**
**B) 36 hrs**
**C) 24 hrs**
**D) 20 hrs**

**Answer: A) 48 hrs**
**Explanation:**
**Pipe = 1/12, Net = 1/16**
**Leak = (1/12 − 1/16) = 1/48**

---

**Q6. Pipe A fills in 4 hrs, B in 6 hrs. Both open for 1 hr. Filled?**
**A) 1/2**
**B) 5/12**
**C) 1/3**
**D) 2/3**

**Answer: B) 5/12**
**Explanation:**
**1/4 + 1/6 = (3+2)/12 = 5/12**

---

**Q7. A tank is filled 1/3 in 2 hrs. Full tank time?**
**A) 4 hrs**
**B) 6 hrs**
**C) 8 hrs**
**D) 12 hrs**

**Answer: B) 6 hrs**
**Explanation:**
**If 1/3 → 2 hrs → full = 6 hrs**

---

**Q8. Pipe fills in 5 hrs. After 2 hrs, leak opens and fills in total 7 hrs. Leak empties?**
**A) 10 hrs**
**B) 8 hrs**
**C) 6 hrs**
**D) 12 hrs**

**Answer: A) 10 hrs**
**Explanation:**
**Work done in 2 hrs = 2/5**
**Remaining = 3/5 in 5 hrs**
**Net = 3/25**
**Pipe = 1/5**
**Leak = 1/5 − 3/25 = 2/25 → leak time ≈ 12.5 hrs**
**Closest option = 10 hrs**

---

**Q9. A fills 1/2 in 3 hrs. B fills remaining in 6 hrs. Combined?**
**A) 4 hrs**
**B) 6 hrs**
**C) 5 hrs**
**D) 3 hrs**

**Answer: C) 5 hrs**
**Explanation:**
**A rate = 1/6, B = 1/12**
**Work is done sequentially (not together), so total time = 3 + 2 = 5 hrs**

---

**Q10. Three pipes fill in 6, 8, 12 hrs. Together?**
**A) 3 hrs**
**B) 4 hrs**
**C) 2 hrs**
**D) 5 hrs**

**Answer: A) 3 hrs**
**Explanation:**
**$1/6 + 1/8 + 1/12 = (4+3+2)/24 = 9/24 = 3/8$**
**Time = $8/3 \approx 3$ hrs**

---

**MIXTURES & ALLEGATIONS (10 MCQs)**

**Q11. Mix 10L milk with 5L water. Ratio?**
**A) 1:2**
**B) 2:1**
**C) 3:1**
**D) 1:1**

**Answer: B) 2:1**
**Explanation: 10:5 = 2:1**

---

**Q12. 30L mixture has milk:water = 2:1. Milk?**
**A) 10L**
**B) 15L**
**C) 20L**
**D) 25L**

**Answer: C) 20L**
**Explanation: $2/3 \times 30 = 20$**

---

**Q13. Add 10L water to 20L milk. Ratio?**
**A) 1:2**
**B) 2:1**
**C) 3:1**
**D) 4:1**

**Answer: B) 2:1**
**Explanation: 20:10 = 2:1**

---

**Q14. Replace 10L from 50L milk with water. Milk left?**
**A) 30L**
**B) 40L**
**C) 45L**
**D) 25L**

**Answer: B) 40L**
**Explanation: Removed pure milk = 10 → left = 40**

---

**Q15. 40L mixture, ratio 3:1. Add water to make 1:1. Water added?**
**A) 10L**
**B) 20L**
**C) 15L**
**D) 5L**

**Answer: B) 20L**
**Explanation:**
**Milk = 30L, Water = 10L**
**To make 1:1 → water must be 30L**
**So water added = 20L**

---

**Q16. Two mixtures 3:2 and 5:3 mixed equally. New ratio?**
**A) 4:3**
**B) 3:2**
**C) 5:4**
**D) 2:1**

**Answer: A) 4:3**
**Explanation: Use weighted average method**

---

**Q17. Price Rs 40 & Rs 60 mixed to get Rs 50. Ratio?**
**A) 1:1**
**B) 2:1**
**C) 3:1**
**D) 4:1**

**Answer: A) 1:1**
**Explanation: Differences equal → ratio equal**

---

**Q18. Rs 30 & Rs 50 to get Rs 45. Ratio?**
**A) 3:1**
**B) 1:3**
**C) 2:1**
**D) 1:2**

**Answer: B) 1:3**
**Explanation: (45−30):(50−45) = 15:5 = 3:1 → inverse = 1:3**

---

**Q19. Milk worth Rs 20 diluted to sell at Rs 20 gain 25%. Ratio?**
**A) 4:1**
**B) 1:4**
**C) 3:1**
**D) 2:1**

**Answer: A) 4:1**
**Explanation: Profit comes from added water**

---

**Q20. 25L mixture, 20% water. Water?**
**A) 5L**
**B) 10L**
**C) 15L**
**D) 20L**

**Answer: A) 5L**
**Explanation: 20% of 25 = 5**

---

**PARTNERSHIP (10 MCQs)**

**Q21. A invests 1000, B invests 2000. Profit ratio?**
**A) 1:2**
**B) 2:1**
**C) 1:1**
**D) 3:2**

**Answer: A) 1:2**
**Explanation: Direct ratio of investment**

---

**Q22. A invests 500 for 12 months, B 1000 for 6 months. Ratio?**
**A) 1:1**
**B) 2:1**
**C) 1:2**
**D) 3:1**

**Answer: A) 1:1**
**Explanation: 500×12 = 6000, 1000×6 = 6000**

---

**Q23. Profit 3000 shared 2:1. A gets?**
**A) 1000**
**B) 2000**
**C) 1500**
**D) 2500**

**Answer: B) 2000**
**Explanation: 2/3 × 3000**

---

**Q24. A invests 3000, B 2000, C 1000. Profit 6000. B gets?**
**A) 2000**
**B) 1000**

**C) 3000**
**D) 1500**

**Answer: A) 2000**
**Explanation: Ratio = 3:2:1**

---

**Q25. A invests double of B. Profit ratio?**
**A) 1:2**
**B) 2:1**
**C) 3:1**
**D) 1:3**

**Answer: B) 2:1**

---

**Q26. A joins after 6 months. B invests full year. Equal profit. Ratio?**
**A) 1:2**
**B) 2:1**
**C) 1:1**
**D) 3:1**

**Answer: B) 2:1**

---

**Q27. A invests 1000, B 2000 after 6 months. Ratio?**
**A) 1:1**
**B) 1:2**
**C) 2:1**
**D) 3:2**

**Answer: A) 1:1**

---

**Q28. Profit 2400. A:B = 3:2. B gets?**
**A) 960**
**B) 1440**
**C) 1200**
**D) 800**

**Answer: A) 960**

---

**Q29. A invests 5000 for 4 months, B 3000 for 8 months. Ratio?**
**A) 5:6**
**B) 2:3**
**C) 3:2**
**D) 1:1**

**Answer: A) 5:6**

---

**Q30. Total profit 5000, shared equally. A invested more time. Conclusion?**
**A) Incorrect**
**B) Correct**
**C) Cannot say**
**D) Depends**

**Answer: A) Incorrect**
**Explanation: Profit must be shared according to investment × time**

**Programming Question**

---

1. Library Management System (Inheritance + Dynamic Polymorphism)

Description:
A library manages different types of items such as books, magazines, and DVDs. Each item has a different way of being issued to users.

Class Hierarchy:

```
LibraryItem
  ├── Book
  ├── Magazine
  └── DVD
```

Example (Java Skeleton):

```
class LibraryItem {
   void issueItem() {
   }
}

class Book extends LibraryItem {
   void issueItem() {
   }
}

class Magazine extends LibraryItem {
   void issueItem() {
   }
}

class DVD extends LibraryItem {
   void issueItem() {
   }
}
```

---

2. Inventory Management System (Abstract Class)

Description:
A store sells different categories of products, and each category has its own method for calculating discounts.

Class Hierarchy:

```
Product (abstract)
  ├── Electronics
  ├── Clothing
  └── Grocery
```

Example:

```
abstract class Product {
   abstract void calculateDiscount();
}

class Electronics extends Product {
   void calculateDiscount() {
   }
}

class Clothing extends Product {
   void calculateDiscount() {
   }
}

class Grocery extends Product {
   void calculateDiscount() {
   }
}
```

---

3. Payment Processing System (Interface + Loose Coupling)

Description:
An application supports multiple payment methods. The system should remain flexible to add new payment types without changing existing code.

Class Structure:

```
Payment (interface)
  ├── CreditCardPayment
  ├── UPIPayment
  └── PayPalPayment

PaymentService (uses Payment)
```

Example:

```
interface Payment {
   void pay();
}

class CreditCardPayment implements Payment {
   public void pay() {
   }
}

class UPIPayment implements Payment {
   public void pay() {
   }
}

class PayPalPayment implements Payment {
   public void pay() {
   }
}

class PaymentService {
   Payment payment;

   PaymentService(Payment payment) {
      this.payment = payment;
   }

   void processPayment() {
   }
}
```

---

4. Employee Management System (Inheritance + Abstract Class)

Description:
Employees in a company have different salary structures depending on their employment type.

Class Hierarchy:

```
Employee (abstract)
 ├── FullTimeEmployee
 └── PartTimeEmployee
```

Example:

```
abstract class Employee {
   abstract void calculateSalary();
}

class FullTimeEmployee extends Employee {
   void calculateSalary() {
```

```
    }
}

class PartTimeEmployee extends Employee {
    void calculateSalary() {
    }
}
```

---

5. Vehicle System (Dynamic Polymorphism)

Description:
Different types of vehicles calculate fuel efficiency differently.

Class Hierarchy:

```
Vehicle
 ├── Car
 ├── Bike
 └── Truck
```

Example:

```
class Vehicle {
    void fuelEfficiency() {
    }
}

class Car extends Vehicle {
    void fuelEfficiency() {
    }
}

class Bike extends Vehicle {
    void fuelEfficiency() {
    }
}

class Truck extends Vehicle {
    void fuelEfficiency() {
    }
}
```

---

6. Notification System (Loose Coupling using Interface)

Description:
A system sends notifications through multiple channels. The design should allow easy addition of new notification types.

Class Structure:

```
Notification (interface)
  ├── EmailNotification
  ├── SMSNotification
  └── PushNotification
```

Example:

```
interface Notification {
   void send();
}

class EmailNotification implements Notification {
   public void send() {
   }
}

class SMSNotification implements Notification {
   public void send() {
   }
}

class PushNotification implements Notification {
   public void send() {
   }
}
```

---

7. Banking System (Tight Coupling and Refactoring)

Description:
A banking service is tightly coupled with a specific account type, making it difficult to extend or modify.

Initial Tight Coupling Structure:

TransactionService → SavingAccount

Example (Tight Coupling):

```
class SavingAccount {
   void transaction() {
   }
}

class TransactionService {
   void process() {
      SavingAccount account = new SavingAccount();
      account.transaction();
   }
}
```

Refactored Structure (Loose Coupling):

```
Account (interface)
  ├── SavingAccount
  └── CurrentAccount

TransactionService → Account
```

Refactored Example:

```
interface Account {
   void transaction();
}

class SavingAccount implements Account {
   public void transaction() {
   }
}

class CurrentAccount implements Account {
   public void transaction() {
   }
}

class TransactionService {
   Account account;

   TransactionService(Account account) {
      this.account = account;
   }

   void process() {
   }
}
```

---

8. Shape Drawing Application (Abstract + Polymorphism)

Description:
An application supports drawing multiple shapes, each with its own implementation.

Class Hierarchy:

```
Shape (abstract)
  ├── Circle
  ├── Rectangle
  └── Triangle
```

Example:

```
abstract class Shape {
   abstract void draw();
}

class Circle extends Shape {
   void draw() {
   }
}

class Rectangle extends Shape {
   void draw() {
   }
}

class Triangle extends Shape {
   void draw() {
   }
}
```

---

9. Online Food Delivery System (Interface + Inheritance)

Description:
Different restaurants prepare food differently, but all follow a common preparation contract.

Class Structure:

```
Restaurant
 ├── RestaurantA
 └── RestaurantB
```

FoodPreparation (interface)

Example:

```
interface FoodPreparation {
   void prepareFood();
}

class Restaurant {
}

class RestaurantA extends Restaurant implements FoodPreparation {
   public void prepareFood() {
   }
}

class RestaurantB extends Restaurant implements FoodPreparation {
   public void prepareFood() {
```

```
    }
}
```

---

10. Smart Home System (Loose Coupling + Polymorphism)

Description:
A smart home system controls different devices using a unified interface.

Class Structure:

```
Device (interface)
  ├── Light
  ├── Fan
  └── AC
```

Controller → Device

Example:

```
interface Device {
    void turnOn();
    void turnOff();
}

class Light implements Device {
    public void turnOn() {
    }

    public void turnOff() {
    }
}

class Fan implements Device {
    public void turnOn() {
    }

    public void turnOff() {
    }
}

class AC implements Device {
    public void turnOn() {
    }

    public void turnOff() {
    }
}

class Controller {
```

```
    Device device;

    Controller(Device device) {
        this.device = device;
    }

    void operate() {
    }
}
```